NeuroSAT: Neuro-symbolic Approaches to the SAT Problem

Reinforcement learning (Alpha(Go)Zero / DQN) with CNNs and Graph Neural Networks for SAT-solver branching heuristics

Donato Meoli

Abstract

We study two neuro-symbolic approaches that learn branching heuristics for a Boolean SAT solver: (i) an *Alpha(Go)Zero*-style method that casts SAT solving as a single-player game and combines Monte Carlo Tree Search (MCTS) with a convolutional policy/value network [1], and (ii) *Graph-Q-SAT*, a value-based reinforcement-learning method that uses a Graph Neural Network (GNN) to approximate the Q-function over a graph representation of the formula [2]. We additionally introduce *GAT-Q-SAT*, a variant that injects Graph Attention layers into the Graph-Q-SAT message passing. The original implementations are several years old and depend on TensorFlow 1.x, GSL and pinned, hard-to-install libraries. We port both to a modern, self-contained PyTorch stack, remove the brittle native dependencies, and *reproduce the original evaluation results exactly.* Our experiments confirm that the attention-augmented model (GAT-Q-SAT) is strongest on structured problems (graph colouring), where a latent sub-structure can be inferred, while on uniform-random 3-SAT near the phase transition the plain Graph-Q-SAT is preferable.

1 Introduction

Boolean satisfiability (SAT) is the canonical NP-complete problem and underlies applications in verification, planning and synthesis. Modern solvers are based on Conflict-Driven Clause Learning (CDCL) and rely on hand-crafted heuristics — most notably VSIDS — to decide which variable to branch on next. The branching heuristic is a natural target for machine learning: a good policy can prune the search tree, especially during the "warm-up" phase where counter-based heuristics make poor decisions.

This project implements and modernises two complementary learning approaches, developed in the context of the Pattern Recognition and Reinforcement Learning courses at the University of Pisa:

- **Reinforcement Learning (Alpha(Go)Zero / MCTS)**, following Wang and Rompf [1];
- **Supervised + Reinforcement Learning (GNN + DQN)**, following Kurin et al. [2], extended here with a Graph Attention Network (GAT) variant [3].

2 Background

SAT and CDCL. A CDCL solver repeatedly (i) *decides* a variable and assigns it a value, (ii) performs unit propagation, and (iii) on a conflict, *learns* a clause and backtracks. The branching heuristic chooses the decision variable; replacing or guiding it with a learned policy is the goal of both methods studied here.

Reinforcement learning. We model the problem as a Markov Decision Process. Graph-Q-SAT uses DQN to approximate the optimal action-value function $Q^\star(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q^\star(s',a')]$ with a target network and experience replay. The Alpha(Go)Zero method instead learns a policy prior π and a value v, and uses MCTS to obtain improved targets by simulation.

3 Method 1 — Alpha(Go)Zero for SAT

Formulation [1]. The CNF formula is represented as a fixed size sparse matrix of *clauses* $\times$ *variables*, where an entry encodes the literal polarity ($(1,0)$ for a positive and $(0,1)$ for a negative occurrence). A simple convolutional network with a policy head π (over the $2 \cdot n_{\text{var}}$ actions "assign variable true/false") and a value head $v \in [-1,1]$ scores states. MiniSat is extended into a game environment that supports MCTS *simulations*: it returns the would-be state for a sequence of branching decisions without irreversibly changing the real state. Self-play collects (state, MCTS visit distribution, outcome) tuples, on which the network is trained with the AlphaZero objective

$$\mathcal{L} = \underbrace{-\sum_a \pi_a^{\text{MCTS}} \log \pi_a}_{\text{cross-entropy}} + \underbrace{(z-v)^2}_{\text{value}} + c\,\|\theta\|_2^2. \tag{1}$$

A fixed-size CNN cannot generalise across problem sizes; this limitation is what motivates the graph-based method below.

Modernisation (this work). The original code is TensorFlow 1.x and its C++ environment depends on GSL. We re-implement the three networks (`model`, `model2`, `model3`) in PyTorch (`models_torch.py`), reproduce the AlphaZero loss and training loop in an `AZTrainer` (`alphazero_torch.py`), and rewrite the self-play / supervised-training orchestration (`main_torch.py`). Crucially, we **remove the GSL dependency**: the Dirichlet exploration noise is reimplemented with the C++11 `<random>` facilities (normalised Gamma$(\alpha_i, 1)$ samples), so the simulation environment builds with nothing but `g++` and `zlib`. The pipeline runs end-to-end on modern Python/PyTorch and on uniform-random 3-SAT (`uf20-91`).

4 Method 2 — Graph-Q-SAT and GAT-Q-SAT

Formulation [2]. The state is a bipartite graph with variable nodes and clause nodes; node features are 2-D one-hot vectors and edge features encode literal polarity, with an empty global attribute. There are two actions per unassigned variable; the agent takes the arg max of the Q-values over variable nodes. The reward is a constant $-p$ per non-terminal step and 0 at termination, encouraging short episodes without reward shaping. The Q-function is an *encode-process-decode* graph network; training uses DQN. Performance is measured by the Median Relative Iteration Reduction (MRIR): the ratio of MiniSat iterations to the model's iterations (values > 1 mean fewer iterations than MiniSat).

GAT-Q-SAT (this work). We add an attention pathway to the core graph block: two `GATConv` layers (with edge features and configurable heads) refine the node embeddings before the global update. Attention lets the model weight neighbouring clauses/variables, which is beneficial precisely when the instance has an exploitable sub-structure.

Modernisation (this work). We port Graph-Q-SAT to the latest PyTorch / PyTorch Geometric. We drop the hard-to-install `torch-scatter`/`torch-sparse` (replaced by `torch_geometric.utils.scatter`), make the gym dependency version-robust (the environment is constructed directly, bypassing the `gym.make` env-checker, and works on `gymnasium`), and add a version-agnostic checkpoint loader that bridges the `GATConv lin_src`/`lin_dst` ↔ `lin` naming across PyTorch Geometric versions. The stack now uses the latest `numpy` (≥ 2); the MiniSat gym extension is rebuilt against it.

5 Experiments

Reproduction (semantic non-regression). We verified that the modernised stack does not change the semantics by re-running the evaluation of the *original trained checkpoints* and comparing against the committed logs. The numbers match **exactly**, e.g. on the graph-colouring set `flat30-60` with a 500-decision cap:

Model	MRIR (median)	mean	committed
Graph-Q-SAT	1.833	2.009	1.833
GAT-Q-SAT	1.667	1.860	1.667

Graph colouring (MRIR, mean over runs). GAT-Q-SAT remains strong and stable across decision caps, including on the larger instances; plain Graph-Q-SAT degrades on large instances at high caps (dropping below 1).

dataset	max10	max50	max100	max300	max500	max1000
GAT-Q-SAT						
flat30-60	1.83	1.83	1.83	1.83	1.83	1.83
flat100-239	1.99	1.99	1.99	1.98	1.98	1.98
flat150-360	2.24	2.32	2.28	2.17	2.14	2.12
flat200-479	2.12	2.11	2.15	1.99	1.90	1.79
Graph-Q-SAT						
flat30-60	1.32	1.13	1.13	1.13	1.13	1.13
flat100-239	1.87	1.79	1.69	1.61	1.60	1.60
flat150-360	2.06	1.75	1.61	1.28	1.20	1.18
flat200-479	1.90	1.95	1.68	1.37	1.10	0.83

Uniform-random 3-SAT. The picture reverses: on random instances near the phase transition (clause/variable ratio ≈ 4.3) the attention model is only mediocre ($\sim$ 1.2–1.8 MRIR), whereas plain Graph-Q-SAT is markedly stronger (e.g. up to $\sim 5\times$ on `uf250-1065`). This supports the interpretation that attention helps when there is a structure to infer (graph colouring) and not on near-uniform random formulas.

The tables above are generated directly from the evaluation logs (`GQSAT/runs/*/*.tsv`) by `aggregate_results.py`, replacing the previous manual spreadsheet workflow.

6 Related Work

The field has converged on graph- and attention-based models for SAT. NeuroSAT learns satisfiability from single-bit supervision [4]; NeuroCore predicts unsat-core variables to refine branching [5];

NeuroComb and **NeuroBack** [6] move inference *offline* (a single query before solving) and, using a Graph Transformer with self-attention, achieve the first wall-clock speed-ups on industrial instances (Kissat, SAT Competition). This trajectory is notable for two reasons relevant to this project: (i) the move to *attention* on SAT graphs corroborates the GAT-Q-SAT direction explored here; and (ii) the known limitation of Graph-Q-SAT — online inference is too slow for wall-clock gains — is exactly what the offline, one-shot inference of NeuroBack overcomes. More recent work continues along the RL-from-algorithm-feedback and restricted-heuristics lines [7], and surveys catalogue the design space [8].

7 Conclusions

We modernised two neuro-symbolic SAT methods to a single, self-contained, latest-version PyTorch stack, removed their brittle native dependencies (TensorFlow 1.x, GSL, `torch-scatter`/`torch-sparse`, pinned gym), and reproduced the original results exactly. The experiments confirm the central qualitative finding: graph attention is advantageous on structured problems (graph colouring) and not on uniform-random SAT. The Alpha(Go)Zero CNN method, while elegant, is fundamentally limited by its fixed-size input and does not generalise across sizes — the gap that the graph-based methods, and the subsequent literature, were designed to close.

References

[1] F. Wang and T. Rompf. *From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha(Go) Zero*. arXiv:1802.05340, 2018.

[2] V. Kurin, S. Godil, S. Whiteson, B. Catanzaro. *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?* NeurIPS 2020. arXiv:1909.11830.

[3] P. Veličković et al. *Graph Attention Networks*. ICLR 2018. arXiv:1710.10903.

[4] D. Selsam et al. *Learning a SAT Solver from Single-Bit Supervision*. arXiv:1802.03685.

[5] D. Selsam and N. Bjørner. *Guiding High-Performance SAT Solvers with Unsat-Core Predictions*. arXiv:1903.04671.

[6] W. Wang et al. *NeuroBack: Improving CDCL SAT Solving using Graph Neural Networks*. ICLR 2024. arXiv:2110.14053.

[7] *Machine Learning for SAT: Restricted Heuristics and New Graph Representations*. arXiv:2307.09141, 2023.

[8] *Neural Approaches to SAT Solving: Design Choices and Interpretability*. arXiv:2504.01173, 2025.